Project Documentation

Libspace

Team Leader: S. Gayathri
Email ID: rajeshwarigayathri9@gmail.com

Team Member: K. Bavana
Email ID: bavanakumar1503@gmail.com

Team Member: R. Garathri
Email ID: garathriramdoss2007@gmail.com

Team Member: S. Deepika
Email ID: deepikasenthil179@gmail.com

PROJECT OVERVIEW

The **ConnectHub** is a backend web application designed to solve the common problem of scattered contact information across multiple platforms. It provides users with a centralized, organized system to store, search, and categorize their professional and personal contacts.

Key Objectives:

- Provide secure user authentication and data isolation
- Enable efficient contact management through CRUD operations
- Implement flexible categorization using tags
- Offer fast search and filtering capabilities
- Deliver a responsive, cross-device user experience

Target Users:

2 Initialize Node Project

Run: `npm init -y`

This will create a `package.json` file.

4 Install Required Dependencies

Install Express:

```
npm install express
npm install mongoose
```

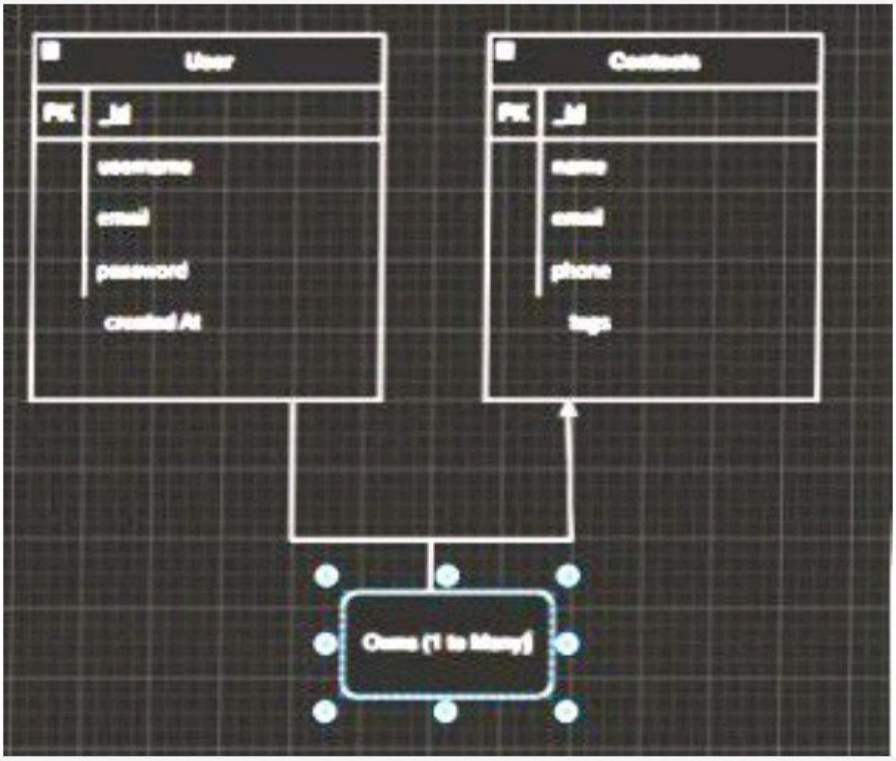
For development (auto-restart server):

```
npm install nodemon --save-dev
```

5 Create Basic Server File

Inside the `server` folder:

- Create a file called: `server.js`



4	Real-time Search	Debounced search across name, email, and tags with instant results

Roles and Responsibility

- User -

1. Registration - Register on the application using the name, email, password
2. Login - Login on the application using the email and registered passw
3. Search - Search for a contact using name , email or tags
4. Add Contact - Add new contact - name , email , phone , notes , tags
5. Edit Contact - Edit the existing contact
6. Delete Contact - Delete any contact

User Flow

```
const contactSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      required: true,
      ref: 'User',
    },
    name: {
      type: String,
      required: [true, 'Please add a contact name'],
      trim: true,
    },
    email: {
      type: String,
      trim: true,
      lowercase: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email',
      ],
    },
    phone: {
      type: String,
      trim: true,
    },
    tags: {
      type: [String],
      default: [],
    },
  },
  {
    timestamps: true,
  }
);
```

Error Handling:

- Implement error handling middleware to catch and handle any errors that occur during the API requests.
- Return appropriate error responses with relevant error messages and HTTP status codes.

```
const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack,
  });
};

const notFound = (req, res, next) => {
  const error = new Error(`Not Found - ${req.originalUrl}`);
  res.status(404);
  next(error);
};

export { errorHandler, notFound };
```

Database Configuration and Setup

1. Configure MongoDB:
   - Install Mongoose.
   - Create database connection.
   - Create Schemas & Models.

2. Connect database to backend:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

```
const PORT = process.env.PORT || 6001;
mongoose.connect(process.env.MONGO_URL, {
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(()=>{


    server.listen(PORT, ()=>{
        console.log(`Running @ ${PORT}`);
    });


}).catch((err)=>{
    console.log("Error: ", err);
})
```

3. Configure Schema:
   - Firstly, configure the Schemas for MongoDB database, to store the data in such pattern
   - Use the data from the ER diagrams to create the schemas.
   - The schemas are looks like for the Application.

```
User.js
server > models > User.js > ...
import mongoose from "mongoose";

const UserSchema = new mongoose.Schema({
    username:{
        type: String,
        require: true
    },
    email:{
        type: String,
        require: true,
        unique: true
    },
    password:{
        type: String,
        require: true
    },
});

const User = mongoose.model("users", UserSchema);
export default User;
```

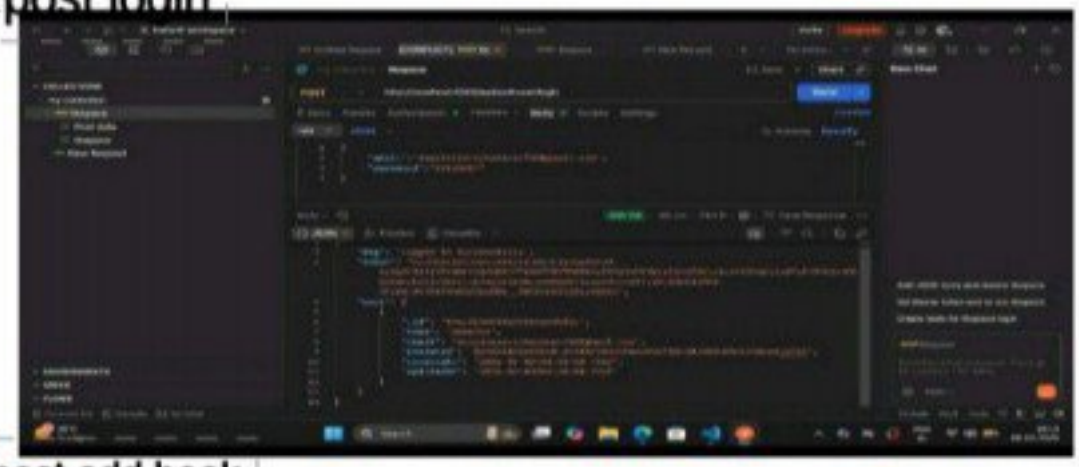
Project Execution

Execution Steps

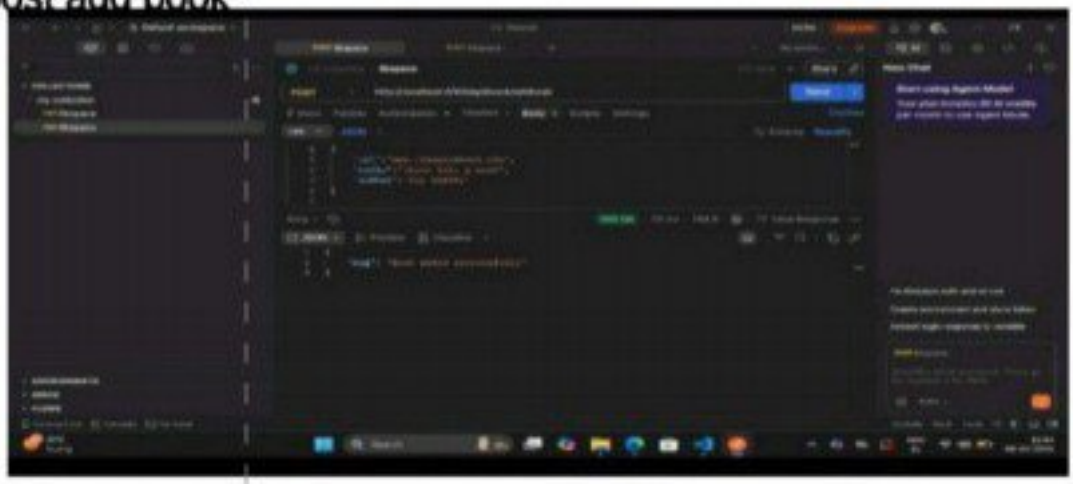
1. Open the project folder in the terminal
2. Make sure you have a start script in the package.js file
3. Use the command **npm start** to start the server
4. You will the **Running on PORT** message on the successfully starting the server
5. You will also see the database connected message if the database is connected successfully
6. Use the Thunder client or Postman for using the created api's

Output Screenshts

1.post login



2post add book



3Get getbook

```
router.get('/search', authentication, searchContacts);
router.route('/')
  .get(authentication, getcontacts)
  .post(authentication, contactValidation, validate, createContact);

router.route('/:id')
  .get(authentication, getContactById)
  .put(authentication, updateContactValidation, validate, updateContact)
  .delete(authentication, deleteContact);
```

Implement Data Models:

- Define Mongoose schemas for the different data entities like user and contact
- Create corresponding Mongoose models to interact with the MongoDB database.
- Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs'
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please add a name'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Please add an email'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email',
      ],
    },
    password: {
      type: String,
      required: [true, 'Please add a password'],
      minlength: 6,
      select: false,
    },
  },
  {
    timestamps: true,
  }
);
```

```
const contactSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      required: true,
      ref: 'User',
    },
    name: {
      type: String,
      required: [true, 'Please add a contact name'],
      trim: true,
    },
    email: {
      type: String,
      trim: true,
      lowercase: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email',
      ],
    },
    phone: {
      type: String,
      trim: true,
    },
    tags: {
      type: [String],
      default: [],
    },
  },
  {
    timestamps: true,
  }
);
```

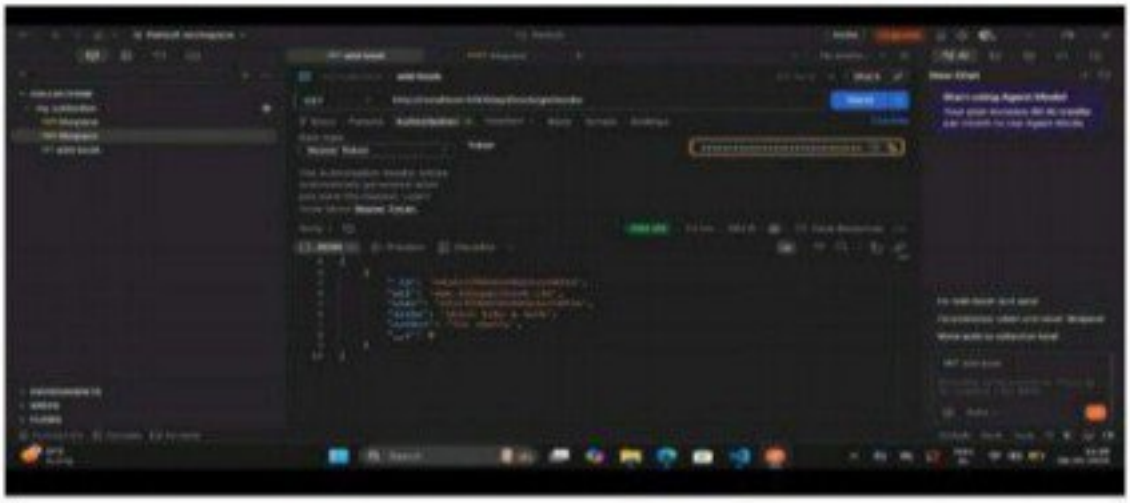
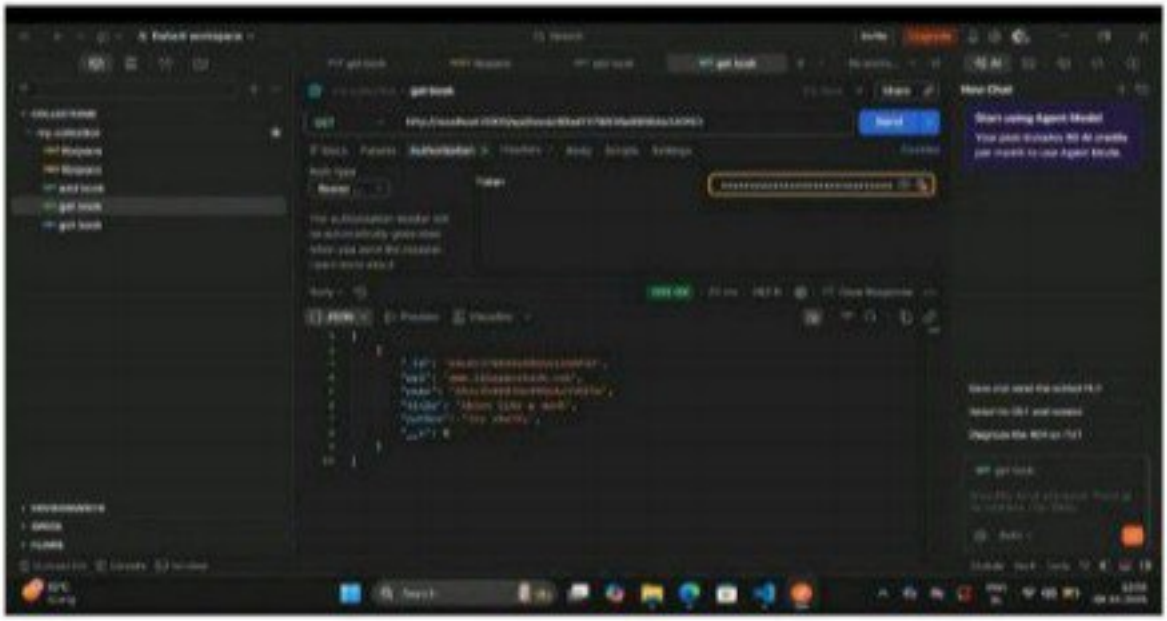
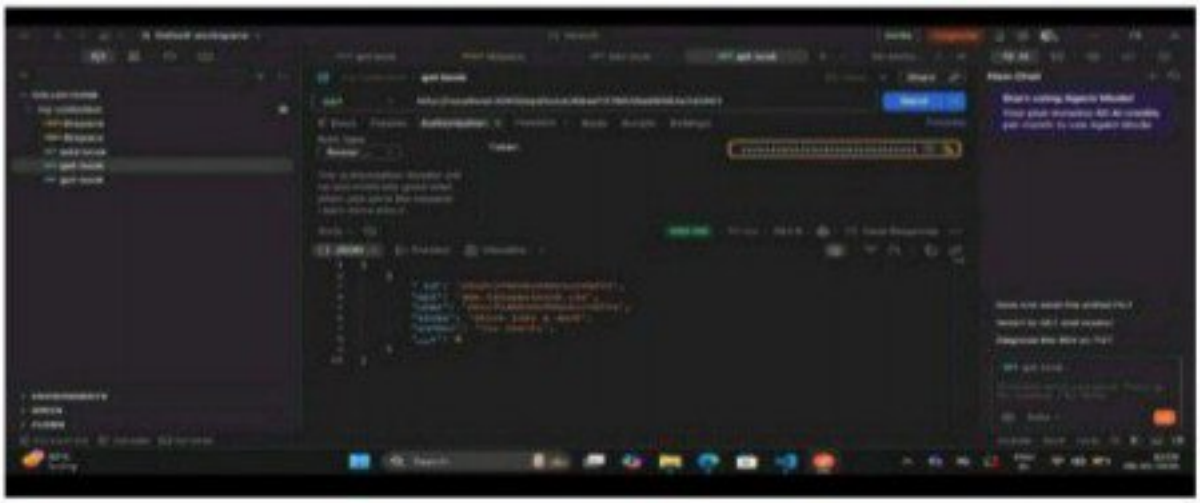
Error Handling:

- Implement error handling middleware to catch and handle any errors that occur during the API requests.
- Return appropriate error responses with relevant error messages and HTTP status codes.

```
const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack,
  });
};

const notFound = (req, res, next) => {
  const error = new Error(`Not Found - ${req.originalUrl}`);
  res.status(404);
  next(error);
};

export { errorHandler, notFound };
```

Database Configuration and Setup